

Lecture 4 Dynamic Programming

Dynamic Programming (DP)

DP approximately is "controlled brute force" and recursion + re-use

Fibonacci Numbers

Use memorized DP algorithms:

```
memo = {}  
fib(n):  
    if n in memo: return memo[n]  
    else: if n <= 2: f=1  
          else f = fib(n-1) + fib(n-2)  
          memo[n] = f  
    return f
```

Only n nonmemorized calls. $\Theta(n)$.

Bottom-up DP algorithm

```
fib = {}  
for k in [1, 2, ..., n]:  
    if k <= 2: f = 1  
    else: f = fib[k-1] + fib[k-2]  
    fib[k] = f  
return fib[n]
```

same computation as memorized DP. $\Theta(n)$.

Faster Algorithm

We have

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

$O(\log n)$ matrix multiplications suffice for computing X^n where X is a 2×2 matrix. Since

$$X^n = \begin{cases} (X^{\lfloor n/2 \rfloor})^2 & \text{if } n \text{ is even} \\ X \cdot (X^{\lfloor n/2 \rfloor})^2 & \text{if } n \text{ is odd} \end{cases}$$

And for each 2×2 matrix it need 8 multiplications and 4 additions.

Chain Matrix Multiplication

Minimum the cost computation for chain matrix multiplication

recurrence formula:

$$m[i, j] = \min\{m[i, k] + m[k + 1, j] + P_{i-1}P_kP_j\}, i < j$$

initially $m[i, j] = 0$ for all $i = j$.

```
for i = 1 to n: C(i,i)=0
for s = 1 to n-1: // s = j - i
  for i = 1 to n-s:
    j = i + s
    recurrence formula for C(i,j)
return C(1,n)
```

Edit Distance

edit distance: the cost of two strings' best alignment. i.e., the minimum number of edits(insert, delete and replace) needed to transform the first string into the second string

S — N O W Y
S U N N — Y

Cost: 3

— S N O W — Y
S U N — — N Y

Cost: 5

Let $dp[i][j]$ denotes the least number of edits to transform the first i characters of A to the first j characters of B .

Base Cases:

- $dp[i][0] = i$: only i deletion.
- $dp[0][j] = j$: only j insertion.

if $A[i] == B[j]$, $dp[i][j] = dp[i-1][j-1]$.

If not,

$$dp[i][j] = 1 + \min \begin{cases} dp[i-1][j], \text{ delete } A[i] \\ dp[i][j-1], \text{ insert } B[j] \\ dp[i-1][j-1], \text{ replace } A[i] \text{ to } B[j] \end{cases}$$

$O(m \times n)$ for two-dimensional array.

Longest Common Subsequence (LCS)

Find a longest subsequence such that the sequence appears in both two strings and have original relative order(not necessary to contagious).

base case:

```
dp[i][0], dp[0][j] = 0
```

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & x_i = y_j \\ \max(dp[i-1][j], dp[i][j-1]), & x_i \neq y_j \end{cases}$$

Knapsack Problem

With Repetition

$$K(w) = \max_{i:w_i \leq w} \{K(w - w_i) + v_i\}$$

```
K(0)=0
```

```
for w = 1 to W:
```

```
    K(w) = max{K(w-w_i) + v_i}, w_i <= w
```

time complexity: $O(nW)$

Without repetition

$$K(w, j) = \max\{K(w - w_j, j - 1) + v_j, K(w, j - 1)\}$$